

Proyecto Final Diso

Memoria del proyecto — Cafetería "La Terminal"

Caso de uso libre — INSO-MAIS Diseño de Software

Sistema de gestión de pedidos para una cafetería, implementado en Java de consola y diseñado en torno a **11 patrones de diseño** combinados.

1. Descripción general del caso de uso

1.1 Contexto

La cafetería "La Terminal" quiere automatizar la toma de pedidos a través de una **terminal de consola** que asista al camarero. Desde una única aplicación se debe poder:

- Crear un pedido vacío e ir añadiendo productos del catálogo.
- Componer productos complejos: bebidas con complementos (azúcar, miel, leches alternativas), pan en distintos formatos (tostada, media tostada, unidad) y platos de *launch* (combinaciones de huevos, bacon, salchichas, yogur, fruta).
- Cobrar el pedido por uno de tres medios: **efectivo**, **Bizum** o **TPV bancario** (este último a través de un dispositivo externo con su propia API).
- Orquestrar el ciclo de vida del pedido: Pendiente → En preparación → Listo → Entregado, propagando los cambios de estado a varios elementos auxiliares (pantalla del cliente, panel informativo, registro de auditoría).
- Ejecutar la preparación de cada producto en la cocina siguiendo una secuencia fija de pasos.

1.2 Alcance

- Aplicación **Java de consola**, sin interfaz gráfica.
- Ejecutable como **un único proceso** y **una única sesión** (sin persistencia).
- Un único local, una única cocina, una única terminal.
- Catálogo de productos cerrado (los productos extensibles se añaden con clases nuevas + entrada en la factory correspondiente).

1.3 Restricciones

- Las **combinaciones inválidas** de toppings (p.ej. leche en un espresso) lanzan `UnsupportedOperationException`.

- Las **transiciones de estado inválidas** (p.ej. entregar un pedido no preparado) lanzan `IllegalStateException`.
- Un único punto de entrada para el cliente (`Terminal`) que solo conoce a la fachada (`CafeteriaManager`).

1.4 Componentes principales identificados

Resumen de alto nivel; los detalles concretos viven en `src/cafeteria/`:

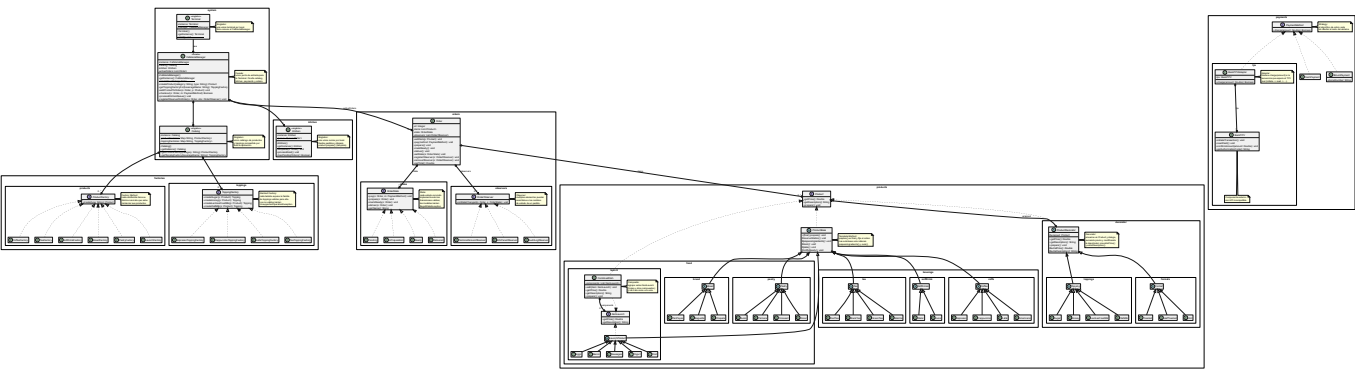
- **Productos** (`cafeteria.products.*`):
 - Componente abstracto: `Product` (interface) y `ProductBase` (clase abstracta con el algoritmo de preparación).
 - Familias concretas: `Coffee`, `Tea`, `SoftDrinks`, `Bread`, `Pastry`, `LaunchProduct` y sus hojas.
 - Composite: `CombinedDish` agrupa varios `ItemLaunch`.
 - Decoradores: `ProductDecorator` con una familias — `Topping` (azúcar, miel, leches).
- **Fábricas** (`cafeteria.factories.*`):
 - `ProductFactory` (interfaz) y sus 6 concretas (una por categoría de producto).
 - `ToppingFactory` (interfaz abstract factory) y sus 4 concretas (una por bebida que admite toppings).
- **Pedido** (`cafeteria.orders.*`):
 - `Order` (sujeto observable y contexto del State).
 - Estados concretos: `Pending`, `InPreparation`, `Ready`, `Delivered`.
 - Observadores concretos: `TerminalScreenObserver`, `InfoPanelObserver`, `AuditLogObserver`.
- **Pago** (`cafeteria.payments.*`):
 - `PaymentMethod` (strategy) y sus implementaciones `CashPayment`, `BizumPayment`, `BankTPVAdapter`.
 - `BankTPV` (adapter externo) con API incompatible (simulada).
- **Sistema** (`cafeteria.system.*` y `cafeteria.kitchen.*`):
 - `Catalog` (singleton, mapa de fábricas).
 - `Kitchen` (singleton, cola de pedidos).
 - `CafeteriaManager` (fachada del subsistema).
 - `Terminal` (singleton, bucle de consola; único cliente de la fachada).

1.5 Patrones aplicados (11)

Template Method · Decorator · Composite · Factory Method · Abstract Factory · State · Observer · Strategy · Adapter · Singleton · Facade.

Cada uno se documenta y ejemplifica en la sección siguiente.

1.6 UML General



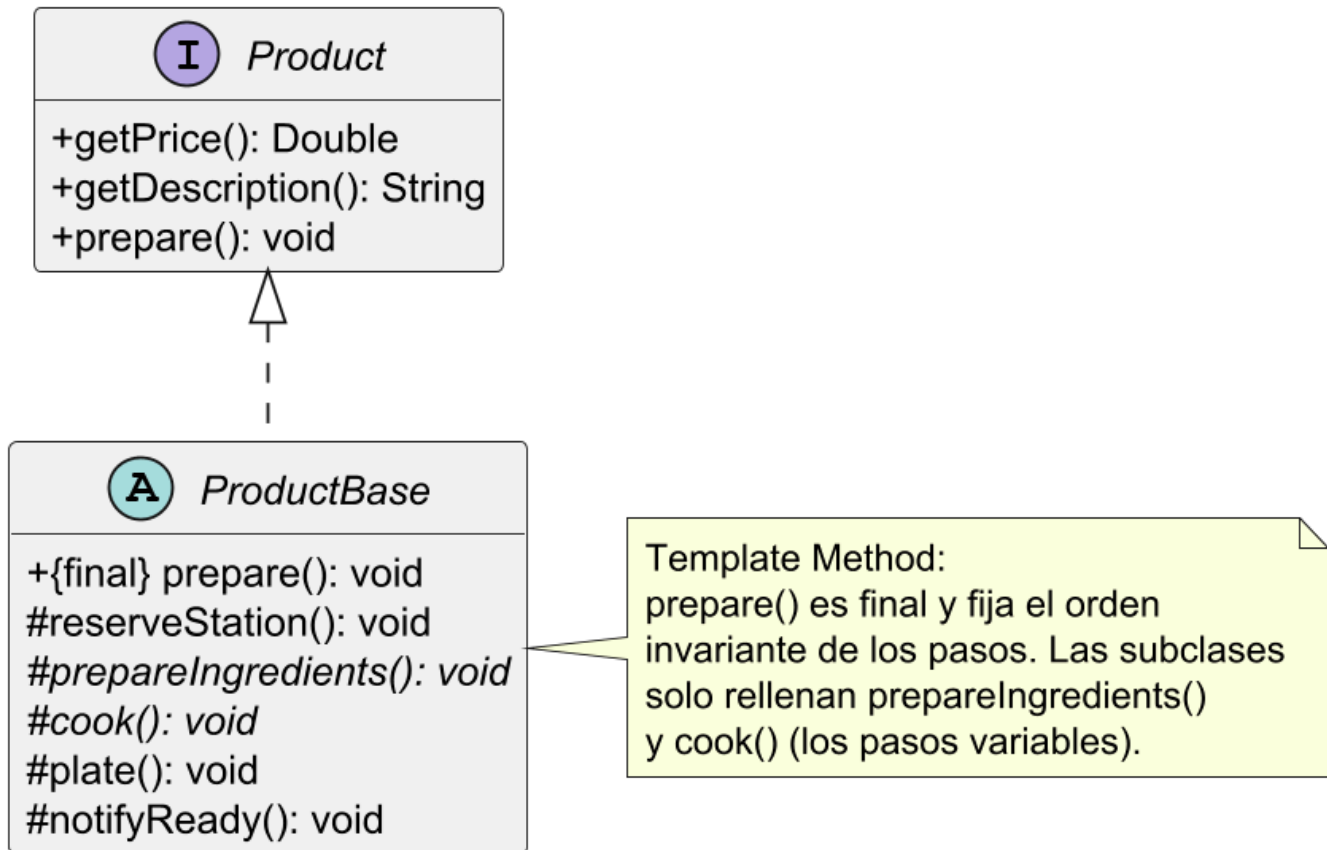
2. Código fuente — un ejemplo por patrón

Para cada patrón se incluye su propósito en el sistema, los roles GoF mapeados a clases del proyecto y **una clase Java representativa** extraída tal cual del repositorio. El resto de hojas (que solo replican el rol) no se reproducen; viven en `src/cafeteria/`.

2.1 Template Method

Propósito: fijar el algoritmo de preparación de un producto (`reserveStation` → `prepareIngredients` → `cook` → `plate` → `notifyReady`) en una superclase, dejando que las subclases solo rellenen los pasos variables.

Rol dentro del patrón	Clase del proyecto
AbstractClass	ProductBase
ConcreteClass	Espresso , RedTea , Croissant , ...



```
package cafeteria.products;

public abstract class ProductBase implements Product{
    private final static Double DEFAULT_PRICE = 100.0;
    private final static String DEFAULT_NAME = "N/A";
    private final String name;
    private Double basePrice;
    protected ProductBase(String name, Double basePrice) {
        this.name = name;
        this.basePrice = basePrice;
    }
    protected ProductBase(String name) {
        this(name, ProductBase.DEFAULT_PRICE);
    }
    protected ProductBase(Double basePrice) {
        this(ProductBase.DEFAULT_NAME, basePrice);
    }
    protected ProductBase() {
        this(ProductBase.DEFAULT_NAME, ProductBase.DEFAULT_PRICE);
    }

    @Override
    public Double getPrice() {
```

```

        return this.basePrice;
    }

    public void setBasePrice(Double price) {
        this.basePrice = price;
    }

    @Override
    public String getDescription() {
        return this.name;
    }

    protected void reserveStation(){
        System.out.println("Se ha inicializado la reserva del producto");
    }

    protected abstract void prepareIngredients();
    protected abstract void cook();
    protected void plate(){
        System.out.println("Se ha emplatado el producto "+this.getDescription());
    }

    protected void notifyReady(){
        System.out.println("El producto " + this.getDescription() + " está listo");
    }

    }

    @Override
    public final void prepare() {
        this.reserveStation();
        this.prepareIngredients();
        this.cook();
        this.plate();
        this.notifyReady();
    }
}

```

El `prepare()` es `final`, garantizando el orden invariante. Las subclases concretas solo proporcionan `prepareIngredients()` y `cook()`.

Clase concreta que implementa el patrón

```

package cafeteria.products.beverage.coffe;

```

```

public class Americano extends Coffee{
private final static String NAME = "Americano";
public Americano() {
    super(Americano.NAME, 1.40);
}

@Override
protected void prepareIngredients() {
    System.out.println("Moliendo café para Americano");
}

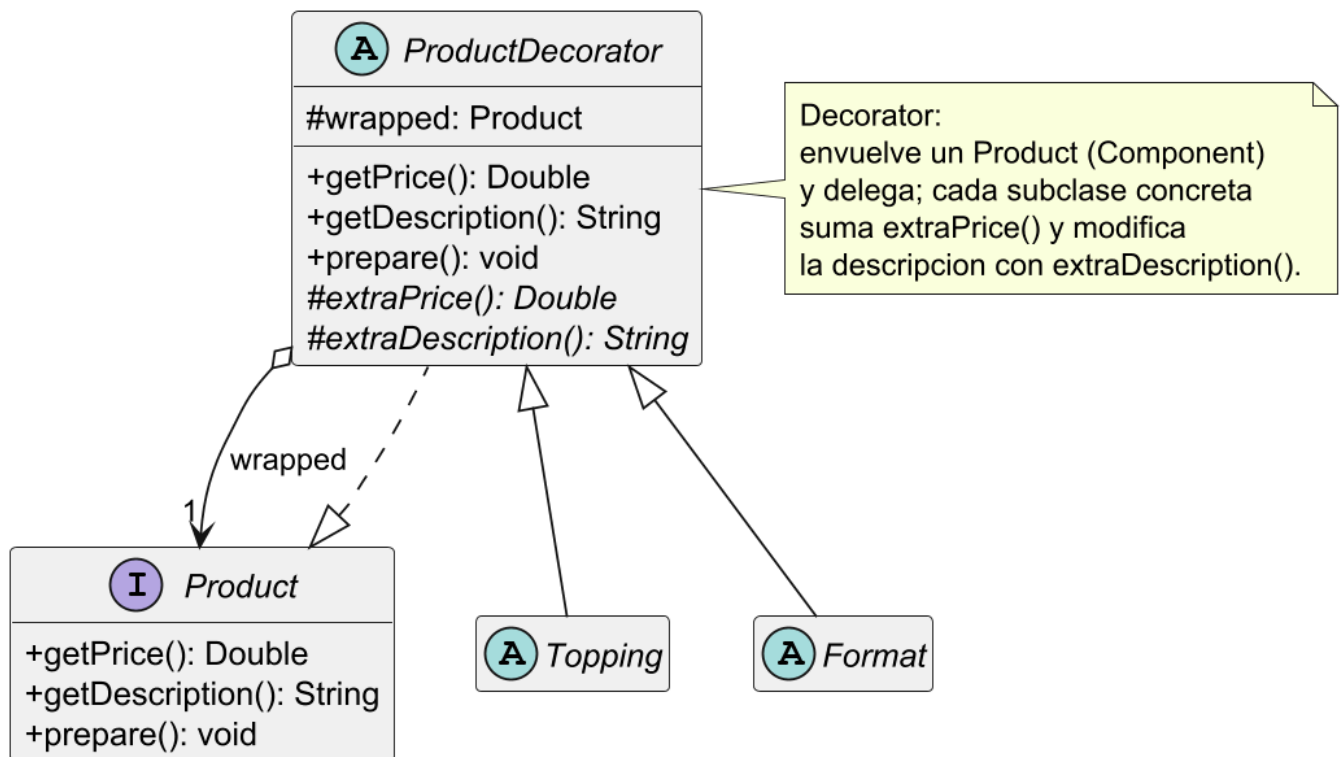
@Override
protected void cook() {
    System.out.println("Extrayendo Americano");
}
}

```

2.2 Decorator

Propósito: añadir responsabilidades (complementos en bebidas) a un `Product` en tiempo de ejecución sin tocar su jerarquía.

Rol dentro del patrón	Clase del proyecto
Component	<code>Product</code>
ConcreteComponent	cualquier subclase de <code>ProductBase</code>
Decorator	<code>ProductDecorator</code>
ConcreteDecorator	<code>Sugar</code> , <code>Honey</code> , <code>LactoseFreeMilk</code> , <code>OatMilk</code>



2.2.1 Component

```

package cafeteria.products;

public interface Product {
    Double getPrice();
    String getDescription();
    void prepare();
}
  
```

2.2.2 ConcreteComponent

```

package cafeteria.products.beverage.tea;

public class GreenTea extends Tea{
    private final static String NAME = "Green Tea";
    public GreenTea() {
        super(GreenTea.NAME, 1.30);
    }

    @Override
    protected void prepareIngredients() {
        System.out.println("Calentando agua para Green Tea");
    }
}
  
```

```

@Override
protected void cook() {
    System.out.println("Infusionando Green Tea");
}
}

```

2.2.3 Decorator

```

package cafeteria.products.decorator;
import cafeteria.products.Product;

public abstract class ProductDecorator implements Product {

    protected final Product wrapped;

    protected ProductDecorator(Product wrapped) {
        this.wrapped = wrapped;
    }

    @Override
    public Double getPrice() {
        return this.wrapped.getPrice() + this.extraPrice();
    }

    @Override
    public String getDescription() {
        return this.wrapped.getDescription() + " " + this.extraDescription();
    }

    @Override
    public void prepare() {
        this.wrapped.prepare();
    }

    protected abstract Double extraPrice();
    protected abstract String extraDescription();
}

```

2.2.4 Concrete decorator

```

package cafeteria.products.decorator.toppings;
import cafeteria.products.Product;

public class LactoseFreeMilk extends Topping {

```

```

private static final String EXTRA_DESCRIPTION = "con leche sin lactosa";

public LactoseFreeMilk(Product wrapped) {
    super(wrapped);
}

@Override
protected Double extraPrice() {
    return 0.30;
}

@Override
protected String extraDescription() {
    return LactoseFreeMilk.EXTRA_DESCRIPTION;
}
}

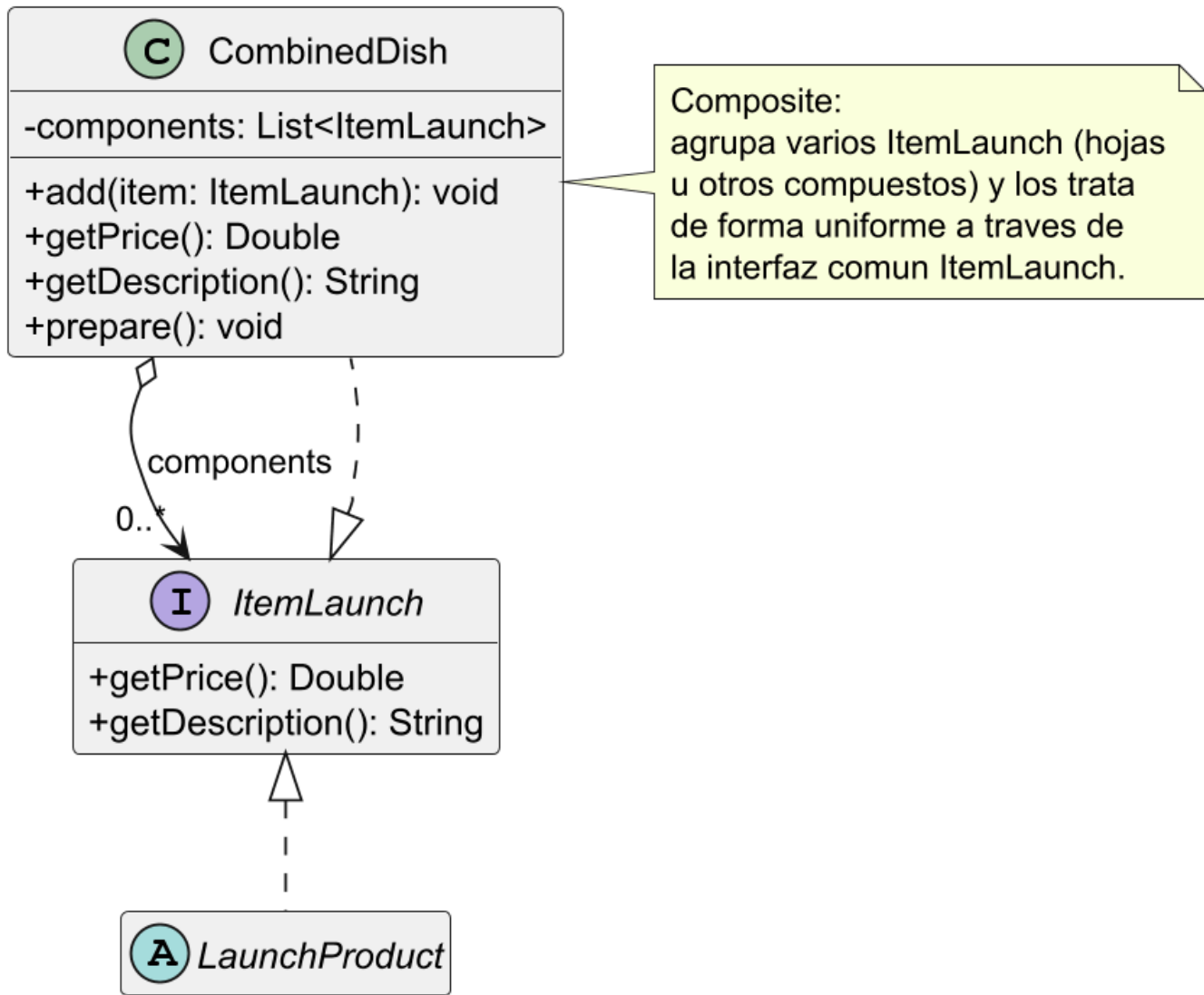
```

Cada decorador concreto solo implementa `extraPrice()` y `extraDescription()`. Los decoradores se pueden anidar (`new Sugar(new OatMilk(new Latte()))`).

2.3 Composite

Propósito: tratar un plato de *launch* (con varios componentes: huevos, bacon, fruta...) y un componente individual con la misma interfaz, permitiendo construir platos combinados recursivamente.

Rol dentro del patrón	Clase del proyecto
Component	ItemLaunch
Leaf	LaunchProduct y sus hojas (Eggs , Bacon , Sausages , Yogurt , Fruit)
Composite	CombinedDish



2.3.1 Component

```

package cafeteria.products.food.launch;

public interface ItemLaunch {
    Double getPrice();
    String getDescription();
}
  
```

2.3.2 Leaf

```

package cafeteria.products.food.launch;

import cafeteria.products.ProductBase;

public abstract class LaunchProduct extends ProductBase implements ItemLaunch {
}
  
```

```

    protected LaunchProduct(String name, Double basePrice) {
        super(name, basePrice);
    }
}

```

```

package cafeteria.products.food.launch;

public class Bacon extends LaunchProduct {
    private static final String NAME = "Bacon";
    public Bacon() {
        super(Bacon.NAME, 1.50);
    }

    @Override
    protected void prepareIngredients() {
        System.out.println("Cortando lonchas de bacon");
    }

    @Override
    protected void cook() {
        System.out.println("Friendo bacon en la plancha");
    }
}

```

2.3.3 Composite

```

package cafeteria.products.food.launch;

import cafeteria.products.Product;
import java.util.ArrayList;
import java.util.List;

public class CombinedDish implements Product, ItemLaunch {

    private final List<ItemLaunch> components;

    public CombinedDish() {
        this.components = new ArrayList<>();
    }

    public void add(ItemLaunch item) {
        this.components.add(item);
    }
}

```

```

@Override
public Double getPrice() {
    Double total = 0.0;
    for (ItemLaunch component : this.components) {
        total += component.getPrice();
    }
    return total;
}

@Override
public String getDescription() {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < this.components.size(); i++) {
        if (i > 0) {
            sb.append(" con ");
        }
        sb.append(this.components.get(i).getDescription());
    }
    return sb.toString();
}

@Override
public void prepare() {
    for (ItemLaunch component : this.components) {
        if (component instanceof Product) {
            ((Product) component).prepare();
        }
    }
}
}
}

```

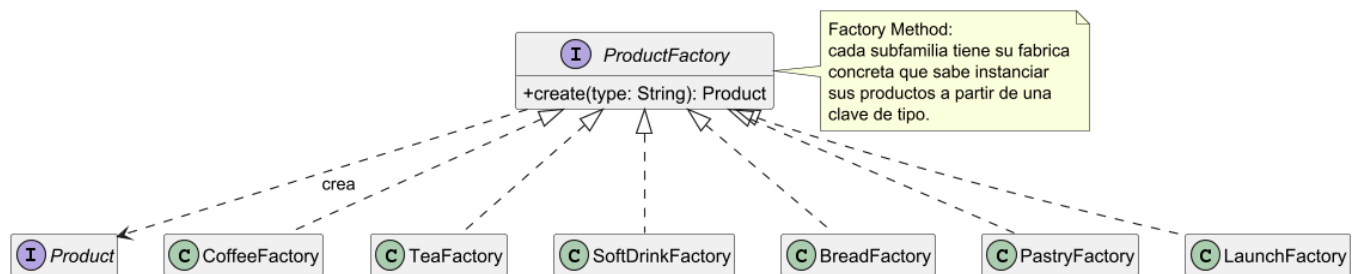
CombinedDish implementa tanto Product como ItemLaunch , de modo que puede usarse en un pedido como producto y a su vez puede contener otros CombinedDish (recursividad).

2.4 Factory Method

Propósito: centralizar la instanciación de cada familia de productos en una clase que conoce las claves de tipo y devuelve el producto correcto.

Rol dentro del patrón	Clase del proyecto
Product	Product
Creator	ProductFactory (interface)

Rol dentro del patrón	Clase del proyecto
ConcreteCreator	CoffeeFactory , TeaFactory , SoftDrinkFactory , BreadFactory , PastryFactory , LaunchFactory



2.4.2 Creator

```

package cafeteria.factories.products;

import cafeteria.products.Product;

public interface ProductFactory {
    Product create(String type);
}

```

2.4.3 ConcreteCreator

```

package cafeteria.factories.products;

import cafeteria.products.Product;
import cafeteria.products.food.pastry.Croissant;
import cafeteria.products.food.pastry.Donut;
import cafeteria.products.food.pastry.Palmera;
import cafeteria.products.food.pastry.Suizo;

public class PastryFactory implements ProductFactory {

    @Override
    public Product create(String type) {
        return switch (type.toLowerCase()) {
            case "croissant" -> new Croissant();
            case "donut" -> new Donut();
            case "palmera" -> new Palmera();
            case "suizo" -> new Suizo();
            default -> throw new IllegalArgumentException("Bollería desconocida: " +
type);
        };
    }
}

```

```

    };
}
}

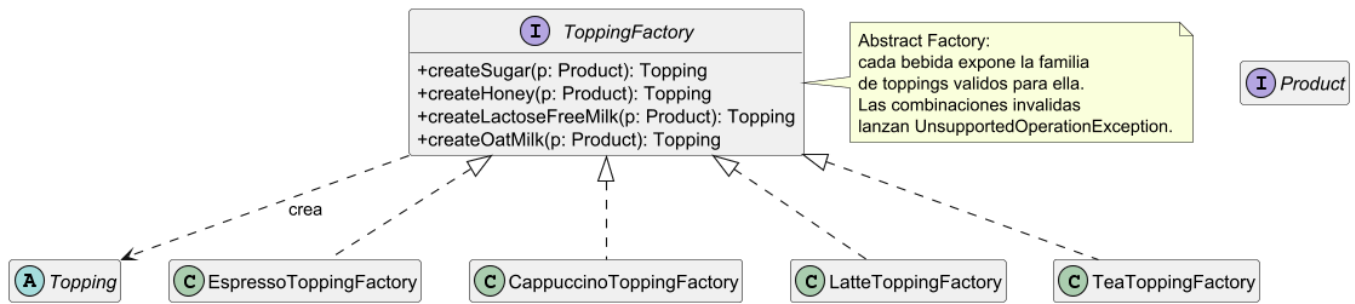
```

El cliente (CafeteriaManager) solicita `factory.create("latte")` sin conocer la clase concreta.

2.5 Abstract Factory

Propósito: exponer una familia coherente de productos relacionados — los **toppings válidos para una bebida concreta**. Las combinaciones que no tienen sentido se rechazan explícitamente.

Rol dentro del patrón	Clase del proyecto
AbstractFactory	ToppingFactory
ConcreteFactory	EspressoToppingFactory , CappuccinoToppingFactory , LatteToppingFactory , TeaToppingFactory
AbstractProduct	Topping
ConcreteProduct	Sugar , Honey , LactoseFreeMilk , OatMilk



2.5.1 AbstractFactory

```

package cafeteria.factories.toppings;

import cafeteria.products.Product;
import cafeteria.products.decorator.toppings.Topping;

public interface ToppingFactory {
    Topping createSugar(Product wrapped);
    Topping createHoney(Product wrapped);
    Topping createLactoseFreeMilk(Product wrapped);
}

```

```
Topping createOatMilk(Product wrapped);  
}
```

2.5.2 ConcreteFactory

```
package cafeteria.factories.toppings;  
  
import cafeteria.products.Product;  
import cafeteria.products.decorator.toppings.Honey;  
import cafeteria.products.decorator.toppings.Sugar;  
import cafeteria.products.decorator.toppings.Topping;  
  
public class EspressoToppingFactory implements ToppingFactory {  
  
    private static final String NOT_ALLOWED = "Complemento no válido para  
espresso";  
  
    @Override  
    public Topping createSugar(Product wrapped) {  
        return new Sugar(wrapped);  
    }  
  
    @Override  
    public Topping createHoney(Product wrapped) {  
        return new Honey(wrapped);  
    }  
  
    @Override  
    public Topping createLactoseFreeMilk(Product wrapped) {  
        throw new  
UnsupportedOperationException(EspressoToppingFactory.NOT_ALLOWED);  
    }  
  
    @Override  
    public Topping createOatMilk(Product wrapped) {  
        throw new  
UnsupportedOperationException(EspressoToppingFactory.NOT_ALLOWED);  
    }  
}
```

Un espresso admite azúcar y miel pero rechaza leche; otras fábricas (LatteToppingFactory) sí la admiten. La fachada elige la fábrica adecuada en función de la bebida.

2.5.3 AbstractProduct

```

package cafeteria.products.decorator.toppings;

import cafeteria.products.Product;
import cafeteria.products.decorator.ProductDecorator;

public abstract class Topping extends ProductDecorator {
    protected Topping(Product wrapped) {
        super(wrapped);
    }
}

```

2.5.4 ConcreteProduct

```

package cafeteria.products.decorator.toppings;

import cafeteria.products.Product;

public class OatMilk extends Topping {
    private static final String EXTRA_DESCRIPTION = "con leche de avena";

    public OatMilk(Product wrapped) {
        super(wrapped);
    }

    @Override
    protected Double extraPrice() {
        return 0.40;
    }

    @Override
    protected String extraDescription() {
        return OatMilk.EXTRA_DESCRIPTION;
    }
}

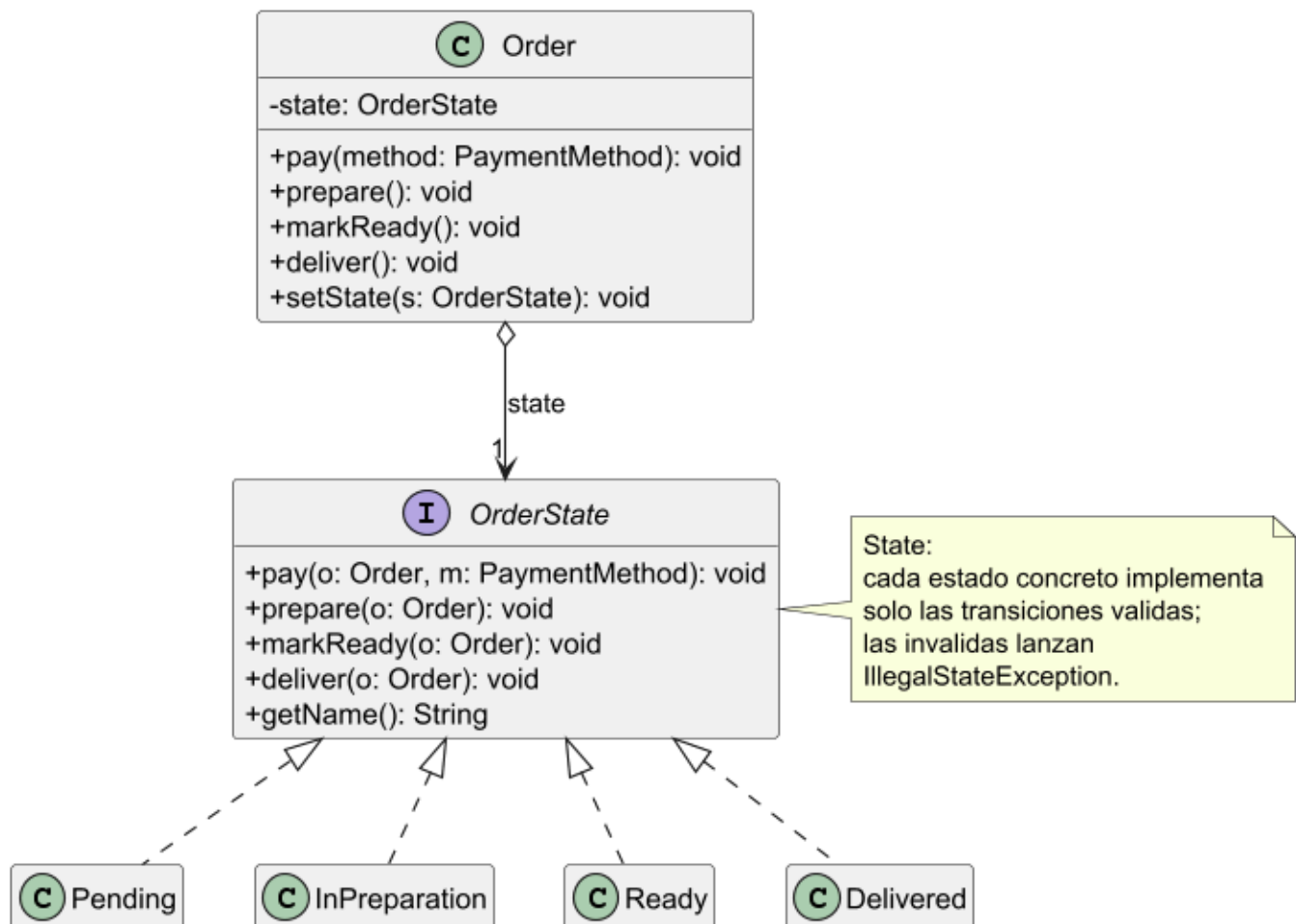
```

2.6 State

Propósito: modelar el ciclo de vida del pedido como objetos de estado intercambiables. Cada estado decide qué transiciones son válidas, evitando un `switch` gigante en `Order`.

Rol dentro del patrón	Clase del proyecto
Context	<code>Order</code>

Rol dentro del patrón	Clase del proyecto
State	OrderState
ConcreteState	Pending , InPreparation , Ready , Delivered



2.6.1 Context

```

package cafeteria.orders;

import cafeteria.orders.observers.OrderObserver;
import cafeteria.orders.states.OrderState;
import cafeteria.orders.states.Pending;
import cafeteria.payments.PaymentMethod;
import cafeteria.products.Product;
import java.util.ArrayList;
import java.util.List;

public class Order {

    private static Integer nextId = 1;
  
```

```
private final Integer id;
private final List<Product> items;
private final List<OrderObserver> observers;
private OrderState state;

public Order() {
    this.id = Order.nextId++;
    this.items = new ArrayList<>();
    this.observers = new ArrayList<>();
    this.state = new Pending();
}

public void addItem(Product product) {
    this.items.add(product);
}

public Double getTotal() {
    Double total = 0.0;
    for (Product p : this.items) {
        total += p.getPrice();
    }
    return total;
}

public Integer getId() {
    return this.id;
}

public String getStateName() {
    return this.state.getName();
}

public void setState(OrderState newState) {
    this.state = newState;
    this.notifyObservers();
}

public void registerObserver(OrderObserver observer) {
    this.observers.add(observer);
}

public void removeObserver(OrderObserver observer) {
    this.observers.remove(observer);
}

private void notifyObservers() {
```

```

        for (OrderObserver o : this.observers) {
            o.onStateChanged(this, this.state);
        }
    }

    public void pay(PaymentMethod method) {
        this.state.pay(this, method);
    }

    public void prepare() {
        this.state.prepare(this);
    }

    public void markReady() {
        this.state.markReady(this);
    }

    public void deliver() {
        this.state.deliver(this);
    }

    public void prepareAllProducts() {
        for (Product p : this.items) {
            p.prepare();
        }
    }
}

```

2.6.2 State

```

package cafeteria.orders.states;

import cafeteria.orders.Order;
import cafeteria.payments.PaymentMethod;

public interface OrderState {
    void pay(Order order, PaymentMethod method);
    void prepare(Order order);
    void markReady(Order order);
    void deliver(Order order);
    String getName();
}

```

2.6.3 ConcreteState

```

package cafeteria.orders.states;

import cafeteria.orders.Order;
import cafeteria.payments.PaymentMethod;

public class Pending implements OrderState {

    private static final String NAME = "Pendiente";
    private static final String INVALID = "El pedido aún no se ha pagado";

    @Override
    public void pay(Order order, PaymentMethod method) {
        Boolean ok = method.charge(order.getTotal());
        if (ok) {
            System.out.println("[Pedido] Pago confirmado, pasando a preparación");
            order.setState(new InPreparation());
        } else {
            System.out.println("[Pedido] Pago rechazado, sigue pendiente");
        }
    }

    @Override
    public void prepare(Order order) {
        throw new IllegalStateException(Pending.INVALID);
    }

    @Override
    public void markReady(Order order) {
        throw new IllegalStateException(Pending.INVALID);
    }

    @Override
    public void deliver(Order order) {
        throw new IllegalStateException(Pending.INVALID);
    }

    @Override
    public String getName() {
        return Pending.NAME;
    }
}

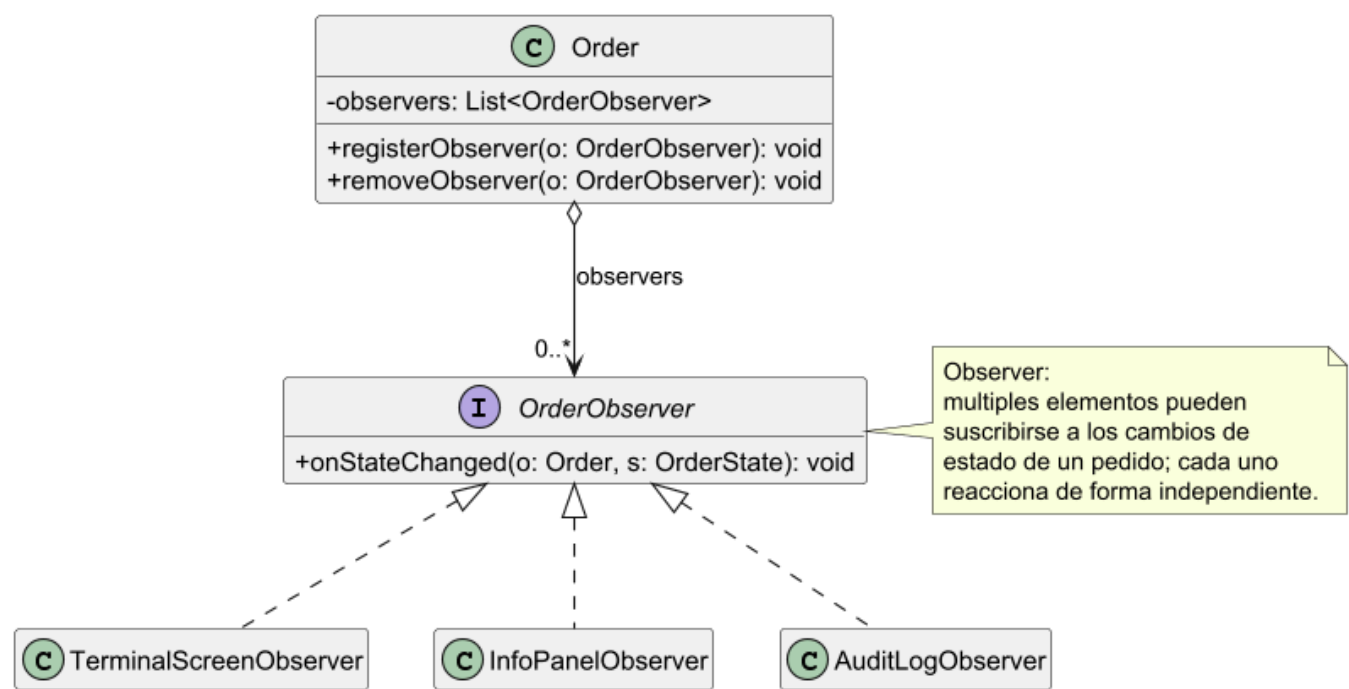
```

Pending solo permite la transición `pay()`. Cualquier otra operación rompe la máquina y lanza `IllegalStateException`. Cada estado concreto se comporta análogamente.

2.7 Observer

Propósito: notificar simultáneamente a varios elementos (pantalla del cliente, panel del local, registro de auditoría) cuando un pedido cambia de estado, sin acoplar `Order` a esas pantallas concretas.

Rol dentro del patrón	Clase del proyecto
Subject	<code>Order</code>
Observer	<code>OrderObserver</code>
ConcreteObserver	<code>TerminalScreenObserver</code> , <code>InfoPanelObserver</code> , <code>AuditLogObserver</code>



2.7.1 Subject

```
package cafeteria.orders;

import cafeteria.orders.observers.OrderObserver;
import cafeteria.orders.states.OrderState;
import cafeteria.orders.states.Pending;
import cafeteria.payments.PaymentMethod;
import cafeteria.products.Product;
import java.util.ArrayList;
import java.util.List;
```

```
public class Order {

    private static Integer nextId = 1;

    private final Integer id;
    private final List<Product> items;
    private final List<OrderObserver> observers;
    private OrderState state;

    public Order() {
        this.id = Order.nextId++;
        this.items = new ArrayList<>();
        this.observers = new ArrayList<>();
        this.state = new Pending();
    }

    public void addItem(Product product) {
        this.items.add(product);
    }

    public Double getTotal() {
        Double total = 0.0;
        for (Product p : this.items) {
            total += p.getPrice();
        }
        return total;
    }

    public Integer getId() {
        return this.id;
    }

    public String getStateName() {
        return this.state.getName();
    }

    public void setState(OrderState newState) {
        this.state = newState;
        this.notifyObservers();
    }

    public void registerObserver(OrderObserver observer) {
        this.observers.add(observer);
    }

    public void removeObserver(OrderObserver observer) {
```

```

        this.observers.remove(observer);
    }

    private void notifyObservers() {
        for (OrderObserver o : this.observers) {
            o.onStateChanged(this, this.state);
        }
    }

    public void pay(PaymentMethod method) {
        this.state.pay(this, method);
    }

    public void prepare() {
        this.state.prepare(this);
    }

    public void markReady() {
        this.state.markReady(this);
    }

    public void deliver() {
        this.state.deliver(this);
    }

    public void prepareAllProducts() {
        for (Product p : this.items) {
            p.prepare();
        }
    }
}

```

2.7.2 Observer

```

package cafeteria.orders.observers;

import cafeteria.orders.Order;
import cafeteria.orders.states.OrderState;

public interface OrderObserver {
    void onStateChanged(Order order, OrderState newState);
}

```

2.7.3 ConcreteObserver

```

package cafeteria.orders.observers;

import cafeteria.orders.Order;
import cafeteria.orders.states.OrderState;

public class TerminalScreenObserver implements OrderObserver {
    @Override
    public void onStateChanged(Order order, OrderState state) {
        if (state != null) {
            System.out.println("[Terminal Screen] " + System.currentTimeMillis() + "
- Pedido #" + order.getId() + " -> " + state.getName());
        } else {
            System.out.println("[Terminal Screen] " + System.currentTimeMillis() + "
- Pedido #" + order.getId() + " -> " + order.getStateName());
        }
    }
}

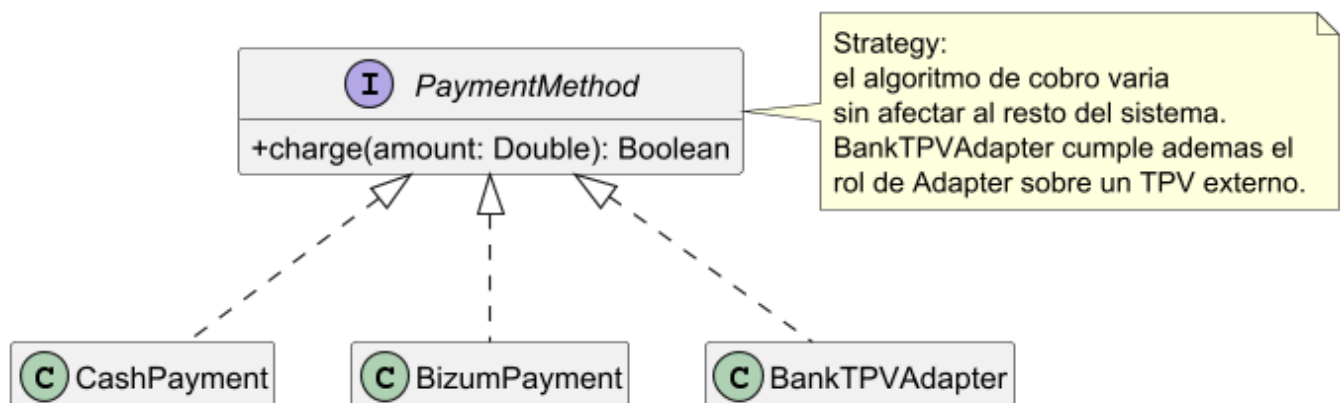
```

Cualquier nuevo "consumidor de eventos" (un KDS de cocina, una pantalla en barra, un *push* al móvil del cliente) se añade con una clase nueva sin tocar `Order`.

2.8 Strategy

Propósito: intercambiar el algoritmo de cobro en caliente. La fachada selecciona la estrategia que indique el cliente y `Order.pay(...)` la invoca sin saber cuál es.

Rol dentro del patrón	Clase del proyecto
Strategy	PaymentMethod
ConcreteStrategy	CashPayment, BizumPayment, BankTPVAdapter
Context	Order



2.8.1 Strategy

```
package cafeteria.payments;

public interface PaymentMethod {
    Boolean charge(Double amount);
}
```

2.8.2 ConcreteStrategy

```
package cafeteria.payments;

public class BizumPayment implements PaymentMethod{
    private final String phoneNumber;

    public BizumPayment(String phoneNumber) {
        this.phoneNumber = phoneNumber;
    }

    @Override
    public Boolean charge(Double amount) {
        System.out.println("Solicitando Bizum de " + amount + " € al teléfono " +
this.phoneNumber);
        System.out.println("Esperando confirmación del cliente");
        System.out.println("Bizum confirmado");
        return true;
    }
}
```

Para añadir un método nuevo (criptomoneda, transferencia, vale) basta con implementar `PaymentMethod`.

2.8.3 Context

```
package cafeteria.orders;

import cafeteria.orders.observers.OrderObserver;
import cafeteria.orders.states.OrderState;
import cafeteria.orders.states.Pending;
import cafeteria.payments.PaymentMethod;
import cafeteria.products.Product;
import java.util.ArrayList;
import java.util.List;
```

```
public class Order {

    private static Integer nextId = 1;

    private final Integer id;
    private final List<Product> items;
    private final List<OrderObserver> observers;
    private OrderState state;

    public Order() {
        this.id = Order.nextId++;
        this.items = new ArrayList<>();
        this.observers = new ArrayList<>();
        this.state = new Pending();
    }

    public void addItem(Product product) {
        this.items.add(product);
    }

    public Double getTotal() {
        Double total = 0.0;
        for (Product p : this.items) {
            total += p.getPrice();
        }
        return total;
    }

    public Integer getId() {
        return this.id;
    }

    public String getStateName() {
        return this.state.getName();
    }

    public void setState(OrderState newState) {
        this.state = newState;
        this.notifyObservers();
    }

    public void registerObserver(OrderObserver observer) {
        this.observers.add(observer);
    }

    public void removeObserver(OrderObserver observer) {
```

```

        this.observers.remove(observer);
    }

    private void notifyObservers() {
        for (OrderObserver o : this.observers) {
            o.onStateChanged(this, this.state);
        }
    }

    public void pay(PaymentMethod method) {
        this.state.pay(this, method);
    }

    public void prepare() {
        this.state.prepare(this);
    }

    public void markReady() {
        this.state.markReady(this);
    }

    public void deliver() {
        this.state.deliver(this);
    }

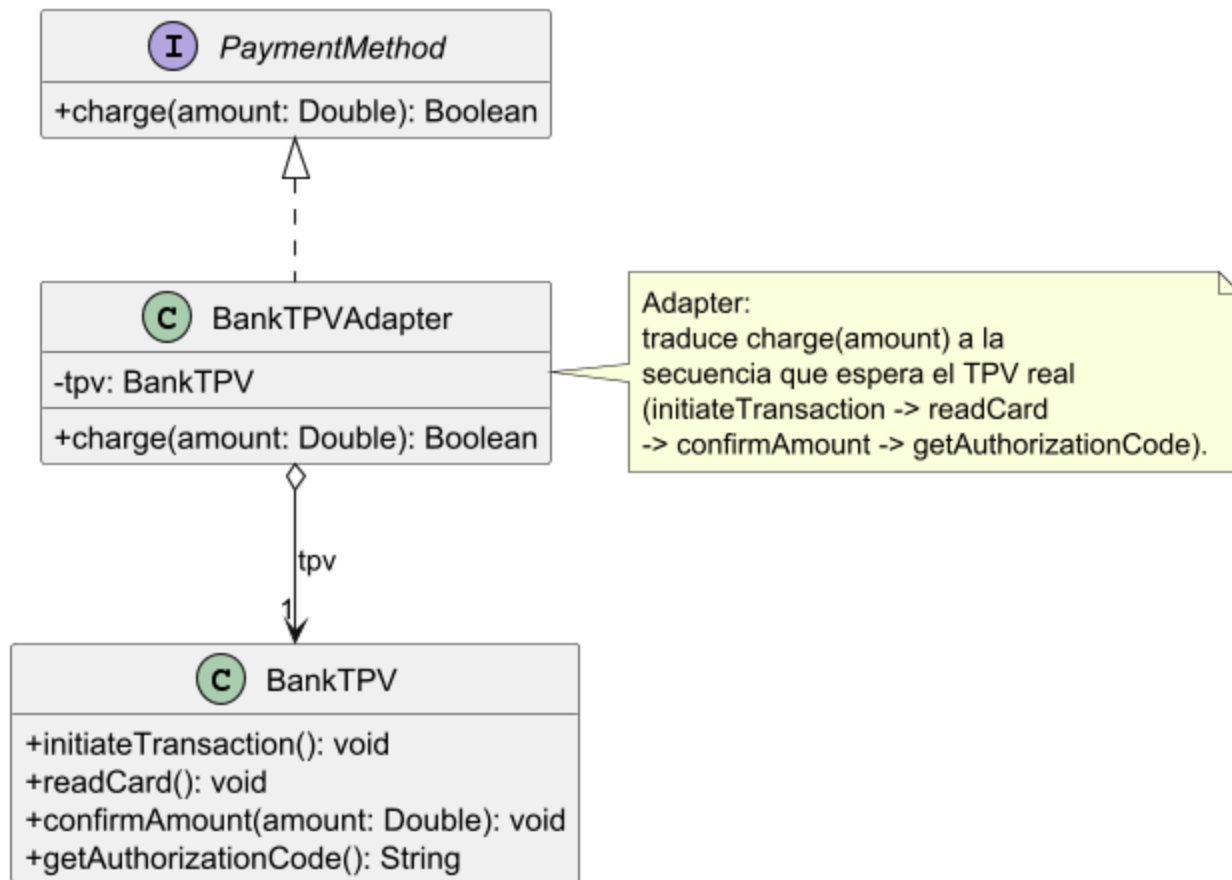
    public void prepareAllProducts() {
        for (Product p : this.items) {
            p.prepare();
        }
    }
}

```

2.9 Adapter

Propósito: integrar un dispositivo externo (BankTPV) cuya API no encaja con PaymentMethod , exponiéndolo como una estrategia más sin modificar ni el dispositivo ni los clientes.

Rol dentro del patrón	Clase del proyecto
Target	PaymentMethod
Adaptee	BankTPV
Adapter	BankTPVAdapter



2.9.1 Target

```

package cafeteria.payments;

public interface PaymentMethod {
    Boolean charge(Double amount);
}

```

2.9.2 Adaptee

Clase que simula un proxy de conexión con una api de pagos con tarjeta

```

package cafeteria.payments.tpv;

//Representa a un componente externo simulando una api de un proveedor externo
// Necesario para poder aplicar el patrón adapter
public class BankTPV {
    private Double currentTransactionAmount;

    public void initiateTransaction() {
        System.out.println("[TPV] Inicializando transacción");
        this.currentTransactionAmount = null;
    }
}

```

```

public void readCard() {
    System.out.println("[TPV] Leyendo banda magnética / chip");
}

public void confirmAmount(Double amount) {
    System.out.println("[TPV] Confirmando importe: " + amount + " €");
    this.currentTransactionAmount = amount;
}

public String getAuthorizationCode() {
    String code = "AUTH-" + System.currentTimeMillis();
    System.out.println("[TPV] Código de autorización: " + code);
    return code;
}
}

```

2.9.3 Adapter

```

package cafeteria.payments.tpv;

import cafeteria.payments.PaymentMethod;

public class BankTPVAdapter implements PaymentMethod {
    private final BankTPV tpv;

    public BankTPVAdapter(BankTPV tpv) {
        this.tpv = tpv;
    }

    @Override
    public Boolean charge(Double amount) {
        this.tpv.initiateTransaction();
        this.tpv.readCard();
        this.tpv.confirmAmount(amount);
        String authCode = this.tpv.getAuthorizationCode();
        System.out.println("Cobro con tarjeta autorizado: " + authCode);
        return true;
    }
}

```

El adaptador traduce el método único `charge(amount)` a la **secuencia** de cuatro llamadas que espera el TPV real.

2.10 Singleton

Propósito: garantizar una única instancia de los recursos globales del local (cocina, catálogo, terminal). No tiene sentido tener dos cocinas concurrentes ni dos catálogos descoordinados.

Rol en el patrón	Clase del proyecto
Singleton	Kitchen , Catalog , Terminal , CafeteriaManager

```
package cafeteria.kitchen;

import cafeteria.orders.Order;
import java.util.LinkedList;
import java.util.Queue;

public class Kitchen {

    private static Kitchen instance;

    private final Queue<Order> queue;

    private Kitchen() {
        this.queue = new LinkedList<>();
    }

    public static Kitchen getInstance() {
        if (Kitchen.instance == null) {
            Kitchen.instance = new Kitchen();
        }
        return Kitchen.instance;
    }

    public void enqueue(Order order) {
        System.out.println("[Cocina] Pedido #" + order.getId() + " encolado");
        this.queue.add(order);
    }

    public void processNext() {
        Order next = this.queue.poll();
        if (next == null) {
            System.out.println("[Cocina] No hay pedidos pendientes");
            return;
        }
        System.out.println("[Cocina] Procesando pedido #" + next.getId());
        next.prepare();
        next.deliver();
    }
}
```

```

public Boolean hasPendingOrders() {
    return !this.queue.isEmpty();
}
}

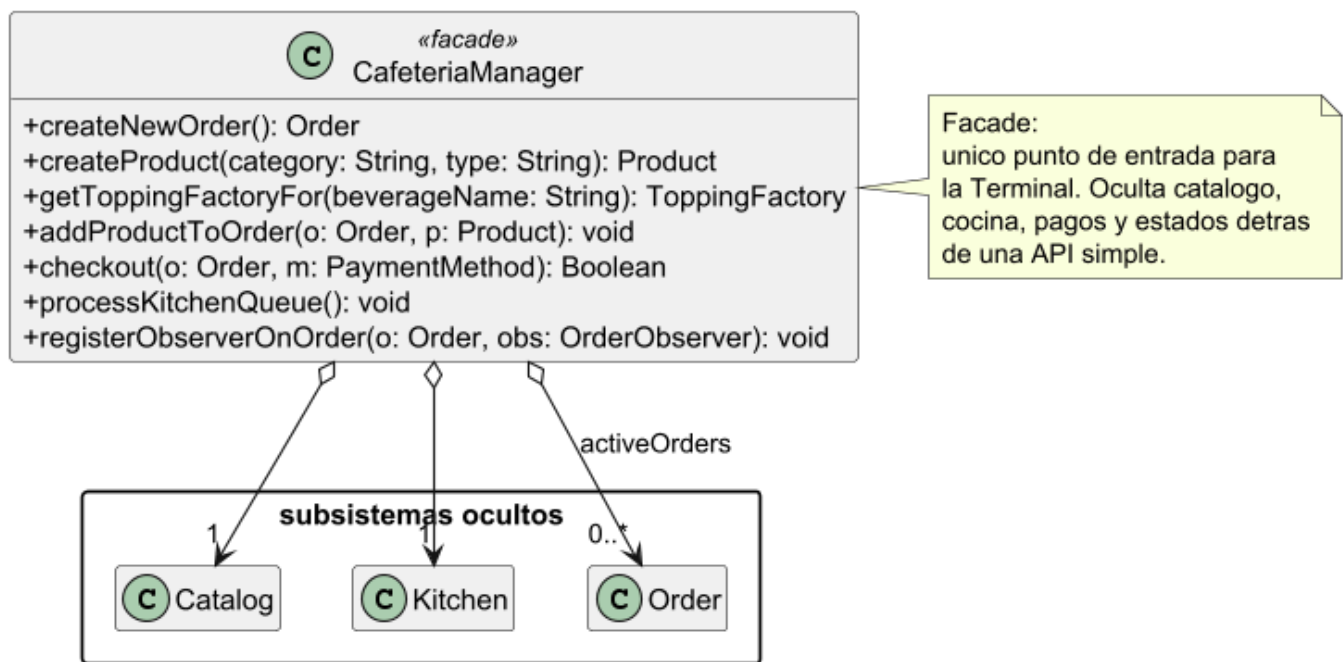
```

Constructor privado + `getInstance()` perezoso. La cola de pedidos vive dentro de la instancia única, garantizando consistencia entre productores (terminal) y consumidor (procesador de cocina).

2.11 Facade

Propósito: ofrecer a `Terminal` un único punto de entrada que oculte la complejidad del sistema (catálogo, fábricas, cocina, pagos, estados, observadores). El cliente no instancia nada manualmente.

Rol dentro del patrón	Clase del proyecto
Facade	<code>CafeteriaManager</code>
Subsystem	<code>Catalog</code> , <code>Kitchen</code> , <code>Order</code> , <code>ProductFactory</code> , <code>ToppingFactory</code> , <code>PaymentMethod</code> , <code>OrderObserver</code>
Client	<code>Terminal</code>



2.11.1 Facade

```

package cafeteria.system;

import cafeteria.factories.products.ProductFactory;
import cafeteria.factories.toppings.ToppingFactory;
import cafeteria.kitchen.Kitchen;
import cafeteria.orders.Order;
import cafeteria.orders.observers.OrderObserver;
import cafeteria.payments.PaymentMethod;
import cafeteria.products.Product;
import java.util.ArrayList;
import java.util.List;

public class CafeteriaManager {

    private static CafeteriaManager instance;

    private final Catalog catalog;
    private final Kitchen kitchen;
    private final List<Order> activeOrders;

    private CafeteriaManager() {
        this.catalog = Catalog.getInstance();
        this.kitchen = Kitchen.getInstance();
        this.activeOrders = new ArrayList<>();
    }

    public static CafeteriaManager getInstance() {
        if (CafeteriaManager.instance == null) {
            CafeteriaManager.instance = new CafeteriaManager();
        }
        return CafeteriaManager.instance;
    }

    public Order createNewOrder() {
        Order order = new Order();
        this.activeOrders.add(order);
        return order;
    }

    public Product createProduct(String category, String type) {
        ProductFactory factory = this.catalog.getProductFactory(category);
        return factory.create(type);
    }

    public ToppingFactory getToppingFactoryFor(String beverageName) {
        return this.catalog.getToppingFactory(beverageName);
    }
}

```

```

}

public void addProductToOrder(Order order, Product product) {
    order.addItem(product);
}

public Boolean checkout(Order order, PaymentMethod method) {
    try {
        order.pay(method);
        this.kitchen.enqueue(order);
        return true;
    } catch (IllegalStateException e) {
        System.out.println("[Manager] Error al cobrar: " + e.getMessage());
        return false;
    }
}

public void processKitchenQueue() {
    while (this.kitchen.hasPendingOrders()) {
        this.kitchen.processNext();
    }
}

public void registerObserverOnOrder(Order order, OrderObserver observer) {
    order.registerObserver(observer);
}
}

```

2.11.2 Subsystem

```

package cafeteria.system;

import cafeteria.factories.products.BreadFactory;
import cafeteria.factories.products.CoffeeFactory;
import cafeteria.factories.products.LaunchFactory;
import cafeteria.factories.products.PastryFactory;
import cafeteria.factories.products.ProductFactory;
import cafeteria.factories.products.SoftDrinkFactory;
import cafeteria.factories.products.TeaFactory;
import cafeteria.factories.toppings.CappuccinoToppingFactory;
import cafeteria.factories.toppings.EspressoToppingFactory;
import cafeteria.factories.toppings.LatteToppingFactory;
import cafeteria.factories.toppings.TeaToppingFactory;
import cafeteria.factories.toppings.ToppingFactory;
import java.util.HashMap;

```

```
import java.util.Map;

public class Catalog {

    private static Catalog instance;

    private final Map<String, ProductFactory> productFactories;
    private final Map<String, ToppingFactory> toppingFactories;

    private Catalog() {
        this.productFactories = new HashMap<>();
        this.productFactories.put("cafe", new CoffeeFactory());
        this.productFactories.put("te", new TeaFactory());
        this.productFactories.put("refresco", new SoftDrinkFactory());
        this.productFactories.put("bolleria", new PastryFactory());
        this.productFactories.put("panaderia", new BreadFactory());
        this.productFactories.put("launch", new LaunchFactory());

        this.toppingFactories = new HashMap<>();
        this.toppingFactories.put("espresso", new EspressoToppingFactory());
        this.toppingFactories.put("cappuccino", new CappuccinoToppingFactory());
        this.toppingFactories.put("latte", new LatteToppingFactory());
        this.toppingFactories.put("te", new TeaToppingFactory());
    }

    public static Catalog getInstance() {
        if (Catalog.instance == null) {
            Catalog.instance = new Catalog();
        }
        return Catalog.instance;
    }

    public ProductFactory getProductFactory(String category) {
        ProductFactory factory =
this.productFactories.get(category.toLowerCase());
        if (factory == null) {
            throw new IllegalArgumentException("Categoría desconocida: " +
category);
        }
        return factory;
    }

    public ToppingFactory getToppingFactory(String beverageName) {
        ToppingFactory factory =
this.toppingFactories.get(beverageName.toLowerCase());
        if (factory == null) {
```

```

        throw new IllegalArgumentException("La bebida no admite complementos: "
+ beverageName);
    }
    return factory;
}
}

```

2.11.3 Client

```

package cafeteria.system;
import cafeteria.factories.toppings.ToppingFactory;
import cafeteria.orders.Order;
import cafeteria.orders.observers.AuditLogObserver;
import cafeteria.orders.observers.InfoPanelObserver;
import cafeteria.orders.observers.TerminalScreenObserver;
import cafeteria.payments.BizumPayment;
import cafeteria.payments.CashPayment;
import cafeteria.payments.PaymentMethod;
import cafeteria.payments.tpv.BankTPV;
import cafeteria.payments.tpv.BankTPVAdapter;
import cafeteria.products.Product;
import cafeteria.products.food.launch.CombinedDish;
import cafeteria.products.food.launch.ItemLaunch;

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Terminal {

    private static Terminal instance;

    private final CafeteriaManager manager;
    private final Scanner scanner;
    private final List<Order> pendingOrders;

    private Terminal() {
        this.manager = CafeteriaManager.getInstance();
        this.scanner = new Scanner(System.in);
        this.pendingOrders = new ArrayList<>();
    }

    public static Terminal getInstance() {
        if (Terminal.instance == null) {
            Terminal.instance = new Terminal();
        }
    }
}

```

```

    }
    return Terminal.instance;
}

public void start() {
    System.out.println("=== Bienvenido a La Terminal ===");
    boolean running = true;
    while (running) {
        System.out.println("\n--- Menú principal ---");
        System.out.println("1) Nueva orden");
        System.out.println("2) Consultar pedidos pendientes");
        System.out.println("3) Salir");
        System.out.print("> ");
        String choice = this.scanner.nextLine().trim();
        switch (choice) {
            case "1": this.newOrder(); break;
            case "2": this.checkPendingOrders(); break;
            case "3": running = false; break;
            default: System.out.println("Opción no válida");
        }
    }
    System.out.println("Hasta luego");
}

private void newOrder() {
    Order order = this.manager.createNewOrder();
    order.registerObserver(new TerminalScreenObserver());
    order.registerObserver(new InfoPanelObserver());
    order.registerObserver(new AuditLogObserver());

    boolean addingItems = true;
    while (addingItems) {
        System.out.println("\n--- Añadir producto ---");
        System.out.println("1) Café (espresso, cappuccino, latte, americano)");
        System.out.println("2) Té (rojo, negro, verde, matcha)");
        System.out.println("3) Refresco (agua, zumo)");
        System.out.println("4) Bollería (croissant, donut, palmera, suizo)");
        System.out.println("5) Panadería (baguette, chapata, panpayes)");
        System.out.println("6) Launch food (plato combinado)");
        System.out.println("7) Terminar y pagar");
        System.out.print("> ");
        String choice = this.scanner.nextLine().trim();

        Product product = null;
        String category = null;
        switch (choice) {

```

```

        case "1": category = "cafe"; break;
        case "2": category = "te"; break;
        case "3": category = "refresco"; break;
        case "4": category = "bolleria"; break;
        case "5": category = "panaderia"; break;
        case "6": category = "launch"; break;
        case "7": addingItems = false; break;
        default: System.out.println("Opción no válida"); continue;
    }

    if (!addingItems) break;

    if (category.equals("launch")) {
        product = this.buildCombinedDish();
        if (product == null) continue;
    } else {
        System.out.print("Tipo: ");
        String type = this.scanner.nextLine().trim();
        try {
            product = this.manager.createProduct(category, type);
        } catch (IllegalArgumentException e) {
            System.out.println("Producto no válido: " + e.getMessage());
            continue;
        }

        if (category.equals("cafe") || category.equals("te")) {
            product = this.applyToppings(product, type, category);
        }
    }

    this.manager.addProductToOrder(order, product);
    System.out.println("Añadido: " + product.getDescription() + " - " +
String.format("%.2f", product.getPrice()));
}

if (order.getTotal() == 0.0) {
    System.out.println("Carrito vacío, cancelando orden");
    return;
}

System.out.println("\nTotal: " + String.format("%.2f", order.getTotal()));
PaymentMethod method = this.choosePaymentMethod();
if (method == null) {
    System.out.println("Pago cancelado");
    return;
}

```

```

        Boolean paid = this.manager.checkout(order, method);
        if (paid) {
            this.pendingOrders.add(order);
            System.out.println("Pedido #" + order.getId() + " encolado");
        }
    }

    private Product applyToppings(Product product, String type, String category)
    {
        String key = category.equals("te") ? "te" : type;
        ToppingFactory factory;
        try {
            factory = this.manager.getToppingFactoryFor(key);
        } catch (IllegalArgumentException e) {
            return product;
        }

        boolean adding = true;
        while (adding) {
            System.out.println("¿Añadir complemento?");
            System.out.println("1) Azúcar  2) Miel  3) Leche sin lactosa  4) Leche
de avena  5) Terminar");
            System.out.print("> ");
            String choice = this.scanner.nextLine().trim();
            try {
                switch (choice) {
                    case "1": product = factory.createSugar(product); break;
                    case "2": product = factory.createHoney(product); break;
                    case "3": product = factory.createLactoseFreeMilk(product); break;
                    case "4": product = factory.createOatMilk(product); break;
                    case "5": adding = false; break;
                    default: System.out.println("Opción no válida");
                }
            } catch (UnsupportedOperationException e) {
                System.out.println("Complemento no permitido: " + e.getMessage());
            }
        }
        return product;
    }

    private PaymentMethod choosePaymentMethod() {
        System.out.println("\n--- Método de pago ---");
        System.out.println("1) Efectivo  2) Bizum  3) Tarjeta  4) Cancelar");
        System.out.print("> ");
        String choice = this.scanner.nextLine().trim();
    }

```

```

switch (choice) {
    case "1": return new CashPayment();
    case "2":
        System.out.print("Número de teléfono: ");
        String phone = this.scanner.nextLine().trim();
        return new BizumPayment(phone);
    case "3": return new BankTPVAdapter(new BankTPV());
    default: return null;
}
}

private void checkPendingOrders() {
    if (this.pendingOrders.isEmpty()) {
        System.out.println("No hay pedidos pendientes");
        return;
    }
    System.out.println("\n--- Pedidos pendientes ---");
    for (Order o : this.pendingOrders) {
        System.out.println("Pedido #" + o.getId() + " - Estado: " +
o.getStateName() + " - Total: " + String.format("%.2f", o.getTotal()));
    }
    System.out.println("\nProcesando cocina...");
    this.manager.processKitchenQueue();
}

private Product buildCombinedDish() {
    CombinedDish dish = new CombinedDish();
    boolean adding = true;

    while (adding) {
        System.out.println("\n--- Componer plato ---");
        System.out.println("Componentes actuales: " +
(dish.getDescription().isEmpty() ? "ninguno" : dish.getDescription()));
        System.out.println("1) Huevos 2) Bacon 3) Salchichas 4) Yogur 5)
Fruta 6) Terminar plato 7) Cancelar");
        System.out.print("> ");
        String choice = this.scanner.nextLine().trim();

        try {
            switch (choice) {
                case "1": dish.add((ItemLaunch) this.manager.createProduct("launch",
"huevos")); break;
                case "2": dish.add((ItemLaunch) this.manager.createProduct("launch",
"bacon")); break;
                case "3": dish.add((ItemLaunch) this.manager.createProduct("launch",
"salchichas")); break;
                case "4": dish.add((ItemLaunch) this.manager.createProduct("launch",

```

```

"yogur")); break;
    case "5": dish.add((ItemLaunch) this.manager.createProduct("launch",
"fruta")); break;
    case "6":
        if (dish.getDescription().isEmpty()) {
            System.out.println("El plato no puede estar vacío");
            continue;
        }
        adding = false;
        break;
    case "7": return null;
    default: System.out.println("Opción no válida");
}
} catch (IllegalArgumentException e) {
    System.out.println("Componente no válido: " + e.getMessage());
}
}

return dish;
}
}

```

`Terminal` solo invoca métodos de `CafeteriaManager`. Toda la maquinaria (singletons, factories, máquina de estados, observadores, cola de cocina) queda encapsulada detrás de esta interfaz.

3. Conclusiones y próximos pasos

3.1 Conclusiones

Aplicar **11 patrones** combinados a un mismo caso de uso ha permitido construir un sistema en el que **cada decisión de diseño tiene un nombre**, y donde cada clase tiene una responsabilidad clara. Agrupando por familias de patrones:

- **Creacionales — Factory Method, Abstract Factory, Singleton.** Aíslan al `CafeteriaManager` del conocimiento concreto de cómo se construye cada producto o complemento, y garantizan una única instancia de los recursos globales (cocina, catálogo, terminal). Añadir un nuevo café o un nuevo topping es sencillo, una clase nueva más una entrada en la factory correspondiente.
- **Estructurales — Decorator, Composite, Adapter, Facade.** Componen comportamiento (decoradores apilables sobre cualquier `Product`), componen estructura (`CombinedDish` agrupa hojas o platos compuestos), integran lo que no encaja (`BankTPVAdapter` adapta un

TPV con API ajena) y simplifican la superficie expuesta al cliente (`CafeteriaManager` oculta seis subsistemas).

- **De comportamiento — Template Method, State, Observer, Strategy.** Modelan los algoritmos del dominio: la secuencia invariante de preparación en cocina (Template Method), el ciclo de vida del pedido con transiciones legales (State), la difusión de cambios a observadores arbitrarios (Observer) y la flexibilidad para sustituir el algoritmo de cobro (Strategy).

El resultado es una arquitectura **abierta a extensión y cerrada a modificación**: cualquier nueva bebida, nuevo medio de pago, nueva pantalla o nuevo estado se introduce mediante una clase nueva sin tocar el núcleo. La complejidad no desaparece, pero queda **descompuesta** en piezas pequeñas y nombradas — siguiendo los principios de desarrollo con patrones de diseño y la poo .

También se evidencian las necesidades de combinar patrones: el `CafeteriaManager` actúa simultáneamente como Facade y Singleton; `BankTPVAdapter` cumple a la vez los roles de Adapter y Strategy. Esta superposición es deliberada y refleja que los patrones describen *intenciones*, no clases distintas.

3.2 Próximos pasos

Mejoras y caminos de evolución para una segunda iteración del proyecto:

- **Persistencia** del histórico de pedidos en JSON o JDBC, para no perder el estado al reiniciar y permitir reportes posteriores.
- **Integración real con TPV** sustituyendo el `BankTPV` mock por una librería bancaria (Redsys, Adyen, Stripe Terminal...). La fachada **no cambiaría**: solo el adapter.
- **Interfaz gráfica** (JavaFX, Swing o una capa web ligera) reutilizando `CafeteriaManager` como API. La fachada está diseñada precisamente para soportar otro cliente además de `Terminal` .
- **Observadores adicionales**: integración con un KDS (*Kitchen Display System*), notificaciones *push* al móvil del cliente, métricas en tiempo real a un panel del encargado.
- **Catalog dinámico**: cargar el mapeo categoría→fábrica desde un fichero de configuración (properties o YAML) para añadir productos sin recompilar.
- **Concurrencia**: hoy la cocina y la terminal son síncronas. Un siguiente paso natural es procesar la cola de cocina en un hilo aparte, con `BlockingQueue` , manteniendo la API pública intacta.
- **CI** con compilación, tests y lint estático (Checkstyle, SpotBugs) ejecutándose en cada commit.

- **Documentación del usuario final:** manual breve de uso de la terminal y guía de extensión para futuros desarrolladores ("cómo añadir un nuevo café en cinco pasos").